

Experiment-9

Aim:

- A. Create a PostgreSQL stored procedure that accepts an employee's gender as an input parameter, counts employees belonging to that gender, returns the count through an OUT parameter, and displays the count using RAISE NOTICE.
 - Table: employee_info (id, name, gender, salary, city)
- B. The procedure accepts a product name and required quantity. It must check available stock. When enough stock exists, it retrieves the product code and price, calculates the total sale amount, inserts an order, decreases remaining stock, and increases quantity sold. Otherwise, no data should be modified.
 - Tables: products and sales

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Theory:

- A Stored Procedure (SP) is a database object that wraps a collection of SQL and procedural statements into a single, reusable unit saved on the server.
- Introduced natively in PostgreSQL 11, stored procedures were specifically engineered to overcome the structural and operational limitations of standard user-defined functions—most notably regarding transaction management.

Code and Output:

A.

```
Experiment_09/postgres@PostgreSQL 18
No limit
Query Query History
1 CREATE TABLE employee_info (
2     id SERIAL PRIMARY KEY,
3     name VARCHAR(50) NOT NULL,
4     gender VARCHAR(10) NOT NULL,
5     salary NUMERIC(10,2) NOT NULL,
6     city VARCHAR(50) NOT NULL
7 );
8
9 INSERT INTO employee_info (name, gender, salary, city)
10 VALUES
11 ('Alok', 'Male', 50000.00, 'Delhi'),
12 ('Priya', 'Male', 60000.00, 'Mumbai'),
13 ('Rajesh', 'Female', 45000.00, 'Bangalore'),
14 ('Sneha', 'Male', 55000.00, 'Chennai'),
15 ('Anil', 'Male', 52000.00, 'Hyderabad'),
16 ('Sunita', 'Female', 48000.00, 'Kolkata'),
17 ('Vijay', 'Male', 47000.00, 'Pune'),
18 ('Ritu', 'Male', 62000.00, 'Ahmedabad'),
19 ('Amit', 'Female', 51000.00, 'Jaipur');
20
Data Output Messages Notifications
INSERT 0 9
Query returned successfully in 92 msec.
```

```
Experiment_09/postgres@PostgreSQL 18
No limit
Query Query History
1 CREATE OR REPLACE PROCEDURE SP_COUNT_GENDER(IN P_GENDER VARCHAR, OUT P_TOTAL_COUNT INT)
2 LANGUAGE PLPGSQL
3 AS
4 $$
5
6 BEGIN
7     SELECT COUNT(id)
8     INTO P_TOTAL_COUNT
9     FROM employee_info
10    WHERE gender = P_GENDER;
11
12     RAISE NOTICE 'The Count of Employee having gender as % are %', P_GENDER, P_TOTAL_COUNT;
13 END;
14 $$;
15
16 CALL SP_COUNT_GENDER('MALE', NULL);
Data Output Messages Notifications
Showing rows: 1 to 1
p_total_count integer
1 0
```

B.

The screenshot shows the PostgreSQL IDE interface. The top toolbar includes icons for file operations, query execution, and settings. The main editor displays the following SQL script:

```
1 CREATE TABLE products (  
2     product_code VARCHAR(10) PRIMARY KEY,  
3     product_name VARCHAR(100) NOT NULL,  
4     price NUMERIC(10,2) NOT NULL,  
5     quantity_remaining INT NOT NULL,  
6     quantity_sold INT DEFAULT 0  
7 );  
8  
9 INSERT INTO products (product_code, product_name, price, quantity_remaining, quantity_sold)  
10 VALUES  
11 ('P001', 'iPhone 13 Pro Max', 109999.00, 10, 0),  
12 ('P002', 'Samsung Galaxy S23 Ultra', 99999.00, 8, 0),  
13 ('P003', 'iPad Air', 55999.00, 5, 0),  
14 ('P004', 'MacBook Pro 14"', 189999.00, 3, 0),  
15 ('P005', 'Sony WH-1000XM5 Headphones', 29999.00, 15, 0);  
16  
17 SELECT * FROM PRODUCTS;
```

The bottom panel shows the 'Data Output' tab with a table of 5 rows:

	product_code [PK] character varying (10)	product_name character varying (100)	price numeric (10,2)	quantity_remaining integer	quantity_sold integer
1	P001	iPhone 13 Pro Max	109999.00	10	0
2	P002	Samsung Galaxy S23 Ultra	99999.00	8	0
3	P003	iPad Air	55999.00	5	0
4	P004	MacBook Pro 14"	189999.00	3	0
5	P005	Sony WH-1000XM5 Headphon...	29999.00	15	0

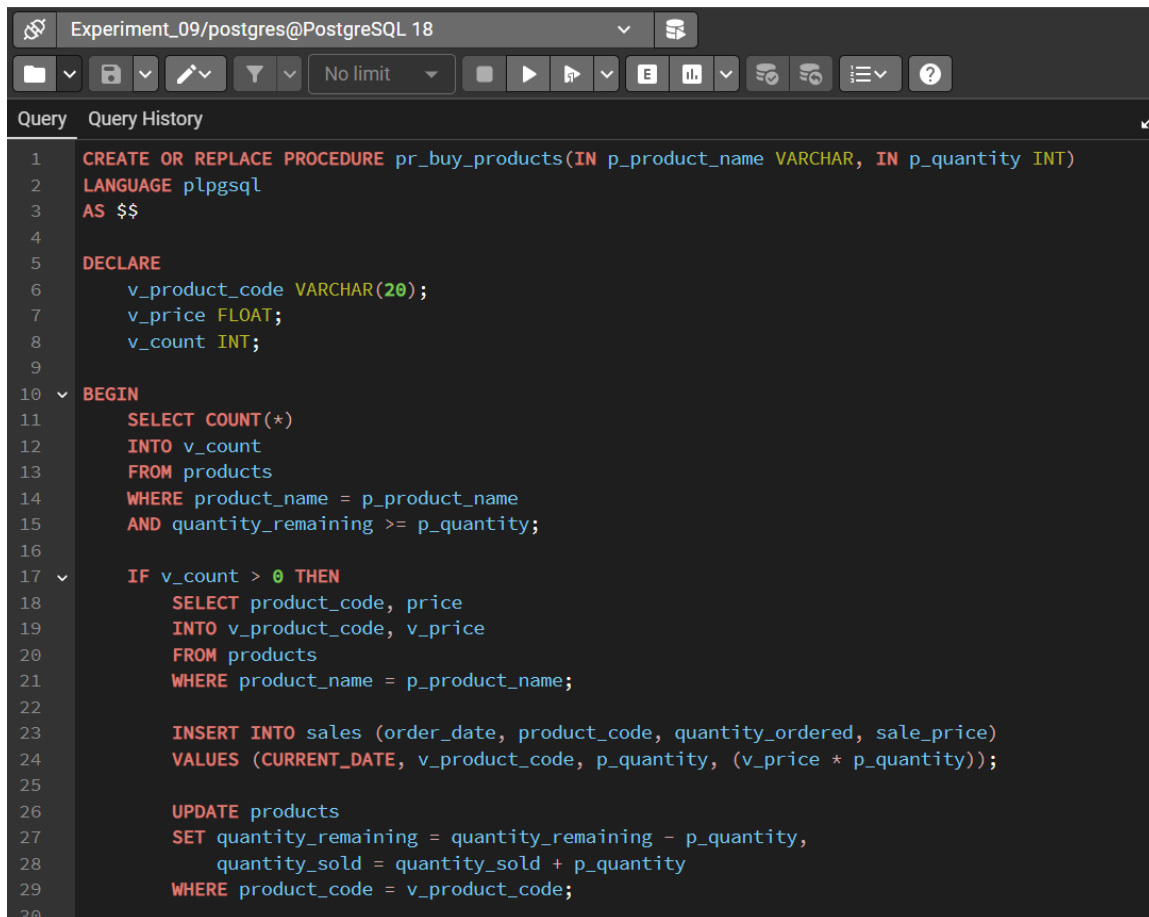
The screenshot shows the PostgreSQL IDE interface. The main editor displays the following SQL script:

```
1 CREATE TABLE sales (  
2     order_id SERIAL PRIMARY KEY,  
3     order_date DATE NOT NULL,  
4     product_code VARCHAR(10) NOT NULL,  
5     quantity_ordered INT NOT NULL,  
6     sale_price NUMERIC(10,2) NOT NULL,  
7     FOREIGN KEY (product_code) REFERENCES products(product_code)  
8 );  
9  
10 INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)  
11 VALUES  
12 ('2025-09-15', 'P001', 1, 109999.00),  
13 ('2025-09-16', 'P002', 2, 199998.00),  
14 ('2025-09-17', 'P003', 1, 55999.00),  
15 ('2025-09-18', 'P005', 2, 59998.00),  
16 ('2025-09-19', 'P001', 1, 109999.00);  
17  
18 SELECT * FROM SALES;
```

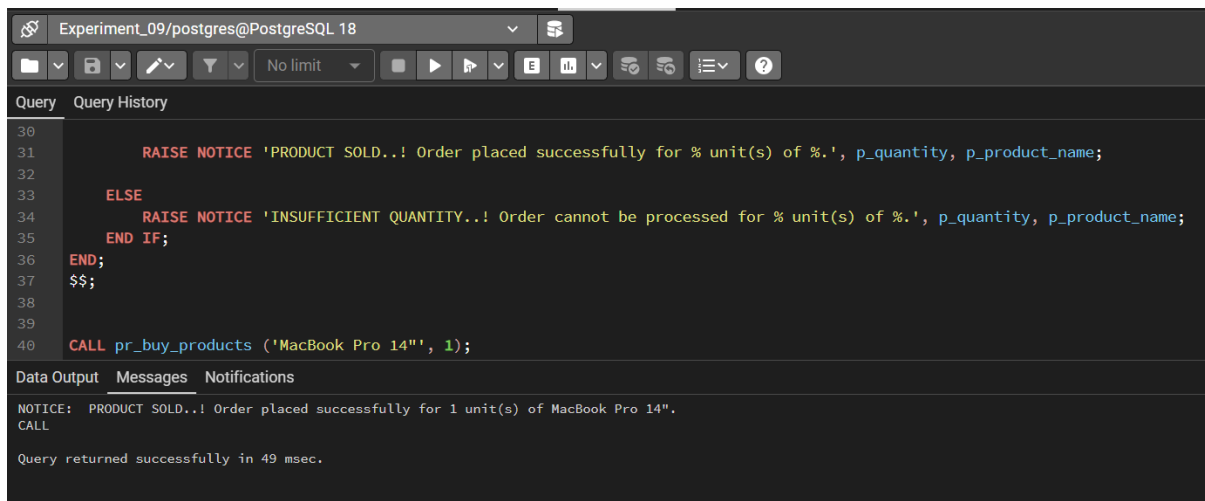
The bottom panel shows the 'Data Output' tab with a table of 5 rows:

	order_id [PK] integer	order_date date	product_code character varying (10)	quantity_ordered integer	sale_price numeric (10,2)
1	1	2025-09-15	P001	1	109999.00
2	2	2025-09-16	P002	2	199998.00
3	3	2025-09-17	P003	1	55999.00
4	4	2025-09-18	P005	2	59998.00
5	5	2025-09-19	P001	1	109999.00

A green status bar at the bottom right indicates: "Successfully run. Total".



```
1 CREATE OR REPLACE PROCEDURE pr_buy_products(IN p_product_name VARCHAR, IN p_quantity INT)
2 LANGUAGE plpgsql
3 AS $$
4
5 DECLARE
6     v_product_code VARCHAR(20);
7     v_price FLOAT;
8     v_count INT;
9
10 BEGIN
11     SELECT COUNT(*)
12     INTO v_count
13     FROM products
14     WHERE product_name = p_product_name
15     AND quantity_remaining >= p_quantity;
16
17     IF v_count > 0 THEN
18         SELECT product_code, price
19         INTO v_product_code, v_price
20         FROM products
21         WHERE product_name = p_product_name;
22
23         INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
24         VALUES (CURRENT_DATE, v_product_code, p_quantity, (v_price * p_quantity));
25
26         UPDATE products
27         SET quantity_remaining = quantity_remaining - p_quantity,
28           quantity_sold = quantity_sold + p_quantity
29         WHERE product_code = v_product_code;
30
```



```
30
31     RAISE NOTICE 'PRODUCT SOLD..! Order placed successfully for % unit(s) of %.', p_quantity, p_product_name;
32
33 ELSE
34     RAISE NOTICE 'INSUFFICIENT QUANTITY..! Order cannot be processed for % unit(s) of %.', p_quantity, p_product_name;
35 END IF;
36 END;
37 $$;
38
39
40 CALL pr_buy_products ('MacBook Pro 14"', 1);
```

Data Output Messages Notifications

NOTICE: PRODUCT SOLD..! Order placed successfully for 1 unit(s) of MacBook Pro 14".
CALL

Query returned successfully in 49 msec.

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with stored procedures in PostgreSQL.